

# Extensibilidad — Plugin System de DocumentIA

## 1. Filosofía & Arquitectura

### ¿Por qué Plugins?

DocumentIA utiliza un sistema **pluggable** para soportar múltiples proveedores de IA sin crear acoplamiento directo. Esto permite:

- **Separation of Concerns:** Lógica de orquestación separada de lógica de proveedores
- **Extensibilidad:** Agregar nuevos proveedores sin modificar código de orquestación
- **Testing:** Implementar providers mock para pruebas sin dependencias externas
- **Fallback:** Intentar múltiples proveedores en cadena si uno falla
- **Configuración por Tipología:** Elegir proveedor dinámicamente según el documento

### Ciclo de Vida del Plugin



### Tipos de Plugins

DocumentIA tiene tres categorías principales:

| Tipo           | Interface               | Métodos               | Responsabilidad                            |
|----------------|-------------------------|-----------------------|--------------------------------------------|
| Extraction     | IExtraerDataProvider    | ObtenerDatosAsync()   | Extraer campos estructurados del documento |
| Classification | IClasificarDataProvider | ClasificarAsync()     | Detectar tipología del documento           |
| Prompt         | IPromptDataProvider     | EjecutarPromptAsync() | Ejecutar prompt libre o análisis custom    |

## 2. Interfaces & Contratos

### IExtraerDataProvider

```
namespace DocumentIA.Functions.Abstractions;

public interface IExtraerDataProvider
{
    /// <summary>
    /// Extrae datos del documento según su tipología.
    /// </summary>
    /// <param name="input">Datos de entrada necesarios para la extracción</param>
    /// <param name="cancellationToken">Token de cancelación</param>
    /// <returns>Resultado de extracción con datos y metadatos</returns>
    Task<ExtraccionResultado> ObtenerDatosAsync(
        ExtraccionInput input,
        CancellationToken cancellationToken = default);
}
```

#### Input Contract:

```
public class ExtraccionInput
{
    public ContratoEntrada Entrada { get; set; } = new();
    public string Tipologia { get; set; } = string.Empty;
    public Dictionary<string, object> DatosNormalizados { get; set; } = new();
    public double? UmbralFallbackEfectivo { get; set; }
    public double? UmbralFallbackEfectivoCompleitud { get; set; }
    public double? UmbralFallbackEfectivoConfianza { get; set; }
    public string? ProviderEfectivo { get; set; }
    public string? ModelKeyEfectivo { get; set; }
    public bool GenerarResumenPorDefecto { get; set; }
}
```

#### Output Contract:

```
public class ExtraccionResultado
{
    public string Proveedor { get; set; } = string.Empty; // "AzureContentUnderstanding"
    public string Modelo { get; set; } = string.Empty; // "doc-intelligence-v3"
    public string? OperationId { get; set; } // Para rastreo
    public int Paginas { get; set; } // Conteo de páginas procesadas
    public bool FallbackUsado { get; set; } // true si se usó fallback
    public string? FallbackRazon { get; set; } // Razón del fallback
    public string? MarkdownExtraido { get; set; } // Contenido en markdown
    public double ConfianzaExtraccion { get; set; } // Confianza global 0-1
    public Dictionary<string, int> TiemposMs { get; set; } = new(); // Tiempos de ejecución
    public Dictionary<string, object> DatosExtraidos { get; set; } = new(); // Campos extraídos
}
```

### IClasificarDataProvider

```
public interface IClasificarDataProvider
{
    Task<ResultadoClasificacion> ClasificarAsync(
        ClasificacionInput input,
        CancellationToken cancellationToken = default);
}
```

#### Input Contract:

```
public class ClasificacionInput
{
    public ContratoEntrada Entrada { get; set; } = new();
    public Dictionary<string, object> DatosNormalizados { get; set; } = new();
    public double? UmbralFallbackEfectivo { get; set; }
    public string? DocumentoBase64Override { get; set; }
    public int CharsTextoNativo { get; set; }
    public int TotalPaginas { get; set; }
    public bool GenerarResumenPorDefecto { get; set; }
}
```

#### Output Contract:

```
public class ResultadoClasificacion
{
    public string TipologiaDetectada { get; set; } = string.Empty;
    public double Confianza { get; set; } // 0-1
    public string Modelo { get; set; } = string.Empty;
    public bool FallbackLLM { get; set; } // true si es fallback
    public Dictionary<string, double>? Scores { get; set; } // Scores por tipología
}
```

## IPromptDataProvider

```
public interface IPromptDataProvider
{
    Task<PromptResultado> EjecutarPromptAsync(
        PromptActivityInput input,
        CancellationToken cancellationToken = default);
}
```

#### Output Contract:

```
public class PromptResultado
{
    public string ResultadoTexto { get; set; } = string.Empty;
    public string? ResultadoJson { get; set; }
    public int TokensUsados { get; set; }
    public Dictionary<string, int> TiemposMs { get; set; } = new();
}
```

## 3. Proveedores Implementados

### A. Extracción (IExtraerDataProvider)

#### Azure Content Understanding (AzureContentUnderstandingProvider)

**Ubicación:** `src/backend/DocumentIA.Functions/Services/AzureContentUnderstandingProvider.cs`

#### Características:

- Extrae datos estructurados usando Azure AI Content Understanding
- Soporta OCR, layout, tablas
- Manejo de throttling con `SemaphoreSlim`
- Circuit breaker para fallos
- Mejor rendimiento para documentos complejos

#### Invocación:

```
// Desde ConfigurableExtraerDataProvider.ObtenerDatosAsync()
if (provider == "azure-content-understanding" || provider == "cu")
{
    return await _azureProvider.ObtenerDatosAsync(input, cancellationToken);
}
```

#### Configuración en ConfiguraciónJson (Tipología):

```
{
  "Extraction": {
    "Provider": "azure-content-understanding",
    "Enabled": true,
    "MinFieldsRatio": 0.75,
    "Model": "doc-intelligence-v3"
  }
}
```

#### Azure Document Intelligence (AzureDocumentIntelligenceExtraerDataProvider)

**Ubicación:** src/backend/DocumentIA.Functions/Services/AzureDocumentIntelligenceExtraerDataProvider.cs

##### Características:

- Extracción con Azure Document Intelligence
- Layout extraction para análisis visual
- Fallback a OCR si Document Intelligence falla
- Más rápido pero menos robusto que CU

#### GPT Direct Extractor (GptDirectExtraerDataProvider)

**Ubicación:** src/backend/DocumentIA.Functions/Services/GptDirectExtraerDataProvider.cs

##### Características:

- Extracción usando Azure OpenAI GPT
- Envía markdown/texto + JSON schema
- Respuesta JSON directo
- Menor costo pero menos preciso

##### Invocación:

```
if (provider == "azure-openai" || provider == "gpt" || provider == "openai")
{
    return await _gptDirectProvider.ObtenerDatosAsync(input, config, cancellationToken);
}
```

#### GPT Fallback Extractor (GptFallbackExtraerDataProvider)

##### Características:

- Se usa como fallback cuando Azure CU falla
- Intenta recuperar extracción usando GPT
- Combina extracción + prompt en una llamada si está habilitado

#### Mock Extractor (MockExtraerDataProvider)

**Ubicación:** src/backend/DocumentIA.Functions/Mocks/MockExtraerDataProvider.cs

**Uso:** Testing y desarrollo local

```

if (provider == "mock")
{
    return await _mockProvider.ObtenerDatosAsync(input, cancellationTokens);
}

```

## B. Clasificación (IClasificarDataProvider)

### GPT Classifier (GptClasificarDataProvider)

**Ubicación:** `src/backend/DocumentIA.Functions/Services/GptClasificarDataProvider.cs`

#### Características:

- Clasificación de documentos usando Azure OpenAI
- 2-phase prompting (fase 1: clasificación, fase 2: validación)
- Soporte para niveles de clasificación customizable
- Integración con ClassificationTipologiaPromptBuilder
- Resumen opcional por defecto

#### Configuración:

```

{
  "Classification": {
    "Provider": "gpt",
    "Model": "gpt-4o",
    "NivelClasificacion": 2
  }
}

```

### Hybrid TDN Classifier (HybridTdnClasificarProvider)

**Ubicación:** `src/backend/DocumentIA.Functions/Services/Classification/HybridTdnClasificarProvider.cs`

#### Características:

- Clasificación híbrida: reglas + layout + DI + LLM
- Extrae clasificadores por tipología de la BD
- Window extraction para análisis de documentos por secciones
- Rescue fallback a GPT si clasificación por reglas falla

#### Flujo:

```

1. RuleBasedTdnClassifier (reglas por tipología)
   ↓ Si falla/confianza baja
2. DocumentWindowExtractor (extrae ventanas)
   ↓
3. FoundryTdnRescueClassifier (GPT rescue)

```

### Azure Document Intelligence Classifier (AzureDocumentIntelligenceClasificarProvider)

#### Características:

- Clasificación usando formularios analizados por DI
- Menos preciso pero más rápido

### Mock Classifier (MockClasificarDataProvider)

**Uso:** Testing

## C. Prompt Execution (IPromptDataProvider)

### OpenAI Prompt Provider (OpenAIPromptDataProvider)

**Ubicación:** `src/backend/DocumentIA.Functions/Services/OpenAIPromptDataProvider.cs`

#### Características:

- Ejecución de prompts libres con OpenAI
- Template interpolation para placeholders ({campo:NombreCampo})
- Soporte JSON output
- Integración con datos extraídos previos

#### Template Example:

```
Analiza este documento:
{contenido}

Extrae:
{campo:InvoiceNumber}
{campo:ClientName}

Contexto:
{datos:PreviousInvoiceNumber}
```

## 4. Anatomía de un Plugin

### Estructura de Directorios

```
src/backend/DocumentIA.Functions/
├─ Abstractions/
│   ├── IExtraerDataProvider.cs      # Interface de extracción
│   ├── IClasificarDataProvider.cs   # Interface de clasificación
│   └─ IPromptDataProvider.cs       # Interface de prompts
├─ Services/
│   ├── AzureContentUnderstandingProvider.cs
│   ├── GptClasificarDataProvider.cs
│   ├── GptDirectExtraerDataProvider.cs
│   ├── ConfigurableExtraerDataProvider.cs # Router de proveedores
│   ├── ConfigurableClasificarDataProvider.cs
│   └─ Classification/
│       ├── HybridTdnClasificarProvider.cs
│       ├── RuleBasedTdnClassifier.cs
│       └─ FoundryTdnRescueClassifier.cs
├─ Mocks/
│   ├── MockExtraerDataProvider.cs
│   └─ MockClasificarDataProvider.cs
└─ Activities/
    ├── ExtraerActivity.cs           # Activity que llama IExtraerDataProvider
    ├── ClasificarActivity.cs        # Activity que llama IClasificarDataProvider
    └─ PromptActivity.cs             # Activity que llama IPromptDataProvider
```

### Clases Requeridas

#### Minimo para un Extraction Plugin:

```

using DocumentIA.Functions.Abstractions;
using DocumentIA.Core.Models;

public class MiExtraerProvider : IExtraerDataProvider
{
    private readonly ILogger<MiExtraerProvider> _logger;

    public MiExtraerProvider(ILogger<MiExtraerProvider> logger)
    {
        _logger = logger;
    }

    public async Task<ExtraccionResultado> ObtenerDatosAsync(
        ExtraccionInput input,
        CancellationToken cancellationToken = default)
    {
        _logger.LogInformation("Extrayendo con MiExtraerProvider para: {Tipologia}", input.Tipologia);

        try
        {
            // 1. Validar input
            if (string.IsNullOrEmpty(input.Tipologia))
                throw new ArgumentException("Tipología requerida");

            // 2. Ejecutar lógica principal
            var datos = await ExecutarExtraccionAsync(input, cancellationToken);

            // 3. Retornar resultado
            return new ExtraccionResultado
            {
                Proveedor = "MiExtractor",
                Modelo = "v1.0",
                Paginas = input.Entrada.Documento.Pages ?? 1,
                DatosExtraidos = datos,
                ConfianzaExtraccion = 0.85,
                FallbackUsado = false,
                TiemposMs = new Dictionary<string, int>
                {
                    { "total", 250 }
                }
            };
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error en extracción");
            throw;
        }
    }

    private async Task<Dictionary<string, object>> ExecutarExtraccionAsync(
        ExtraccionInput input,
        CancellationToken cancellationToken)
    {
        // Tu lógica aquí
        return new Dictionary<string, object>
        {
            { "campo1", "valor1" },
            { "campo2", "valor2" }
        };
    }
}

```

## Dependency Injection Setup

Registrar en Program.cs :

```
// En Program.cs BuildHost() → ConfigureServices()

// 1. Registrar dependencias internas si las hay
services.AddScoped<IMiDependencia, MiDependencia>();

// 2. Registrar provider como Singleton (stateless) o Scoped
services.AddSingleton<MiExtraerProvider>();

// 3. Si es el default, registrar como IExtraerDataProvider
// (Nota: DocumentIA usa ConfigurableExtraerDataProvider como router)
// NO registrar directamente como interfaz si usas Configurable router
```

## 5. Step-by-Step: Crear un Nuevo Plugin

### Paso 1: Crear Interface Implementation

```
// File: src/backend/DocumentIA.Functions/Services/MiCustomExtraerProvider.cs

using DocumentIA.Functions.Abstractions;
using DocumentIA.Core.Models;
using DocumentIA.Core.Configuration;
using Microsoft.Extensions.Logging;

namespace DocumentIA.Functions.Services;

public class MiCustomExtraerProvider : IExtraerDataProvider
{
    private readonly ILogger<MiCustomExtraerProvider> _logger;
    private readonly TipologiaConfigLoader _configLoader;

    public MiCustomExtraerProvider(
        ILogger<MiCustomExtraerProvider> logger,
        TipologiaConfigLoader configLoader)
    {
        _logger = logger;
        _configLoader = configLoader;
    }

    public async Task<ExtraccionResultado> ObtenerDatosAsync(
        ExtraccionInput input,
        CancellationToken cancellationToken = default)
    {
        // Ver Paso 5: Implementación completa
        throw new NotImplementedException();
    }
}
```

### Paso 2: Inyectar en Activity

**Ya hecho:** Las Activities ya reciben `ConfigurableExtraerDataProvider` que enruta automáticamente.

```
// No necesitas modificar ExtraerActivity.cs
// ConfigurableExtraerDataProvider lo hará automáticamente
// si registras tu provider en Program.cs
```

### Paso 3: Registrar en DI Container



```
// Program.cs, línea ~160

services.AddSingleton<MiCustomExtraerProvider>();
```

#### Paso 4: Agregar Enrutamiento en ConfigurableExtraerDataProvider

```
// Archivo: ConfigurableExtraerDataProvider.cs

public async Task<ExtraccionResultado> ObtenerDatosAsync(ExtraccionInput input, ...)
{
    var provider = input.ProviderEfectivo ?? config.Extraction.Provider;

    return provider.ToLowerInvariant() switch
    {
        "azure-content-understanding" => await _azureProvider.ObtenerDatosAsync(input, cancellationTokens),
        "gpt" => await _gptDirectProvider.ObtenerDatosAsync(input, config, cancellationTokens),
        "mi-custom" => await _miCustomProvider.ObtenerDatosAsync(input, cancellationTokens), // ← AGREGAR
        _ => throw new NotSupportedException($"Proveedor '{provider}' no soportado")
    };
}
```

Agregar inyección en constructor:

```
public ConfigurableExtraerDataProvider(
    // ... otros parámetros
    MiCustomExtraerProvider miCustomProvider, // ← AGREGAR
    // ...
)
{
    // ...
    _miCustomProvider = miCustomProvider;
}
```

#### Paso 5: Implementación Completa

```

public class MiCustomExtraerProvider : IExtraerDataProvider
{
    private readonly ILogger<MiCustomExtraerProvider> _logger;
    private readonly TipologiaConfigLoader _configLoader;
    private readonly IBlobStorageService _blobService;

    public MiCustomExtraerProvider(
        ILogger<MiCustomExtraerProvider> logger,
        TipologiaConfigLoader configLoader,
        IBlobStorageService blobService)
    {
        _logger = logger;
        _configLoader = configLoader;
        _blobService = blobService;
    }

    public async Task<ExtraccionResultado> ObtenerDatosAsync(
        ExtraccionInput input,
        CancellationToken cancellationToken = default)
    {
        var sw = System.Diagnostics.Stopwatch.StartNew();
        var config = _configLoader.LoadConfig(input.Tipologia);

        try
        {
            // 1. Validar
            if (string.IsNullOrEmpty(input.Tipologia))
                throw new ArgumentException("Tipología requerida");

            // 2. Descargar documento
            var documento = await _blobService.DescargarAsync(
                input.Entrada.Documento.Key, cancellationToken);

            // 3. Procesar con tu lógica
            var campos = await ExtraerCamposAsync(documento, input.Tipologia, cancellationToken);

            // 4. Mapear resultado
            var resultado = new ExtraccionResultado
            {
                Proveedor = "MiCustom",
                Modelo = "v1.0",
                Paginas = input.Entrada.Documento.Pages ?? 1,
                MarkdownExtraido = null,
                ConfianzaExtraccion = 0.80,
                FallbackUsado = false,
                DatosExtraidos = campos,
                TiemposMs = new Dictionary<string, int>
                {
                    { "download", 100 },
                    { "extraction", 300 },
                    { "total", sw.ElapsedMilliseconds }
                }
            };

            _logger.LogInformation(
                "Extracción exitosa para {Tipologia}. Campos={Campos}, Tiempo={Tiempo}ms",
                input.Tipologia,
                campos.Count,
                sw.ElapsedMilliseconds);

            return resultado;
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error extrayendo documento para {Tipologia}", input.Tipologia);
            throw;
        }
    }
}

```

```
private async Task<Dictionary<string, object>> ExtraerCamposAsync(
    byte[] documento,
    string tipologia,
    CancellationToken cancellationToken)
{
    // Tu lógica de extracción aquí
    // Ejemplo: parsear PDF, OCR, llamar API, etc.

    await Task.Delay(300, cancellationToken); // Simular trabajo

    return new Dictionary<string, object>
    {
        { "InvoiceNumber", "INV-2024-001" },
        { "ClientName", "Acme Corp" },
        { "Amount", 1500.00 }
    };
}
}
```

## 6. Integración con Orquestación

### Cómo se Invoca desde Activity Function

```
DocumentProcessOrchestrator
├─ ResolverTipologiaActivity
│   └─ Retorna: TipologiaEntity (con ConfiguracionJson)
├─ ExtraerActivity (si habilitado)
│   ├── Input: ExtraccionInput (con Tipologia, DatosNormalizados, Provider efectivo)
│   ├── IExtraerDataProvider.ObtenerDatosAsync()
│   │   └─ Llama: ConfigurableExtraerDataProvider
│   │       └─ Enruta a: Tu MiCustomExtraerProvider
│   └─ Output: ExtraccionResultado
├─ ClasificarActivity
│   ├── Input: ClasificacionInput
│   ├── IClasificarDataProvider.ClasificarAsync()
│   └─ Output: ResultadoClasificacion
└─ PromptActivity (si habilitado)
    ├── Input: PromptActivityInput
    ├── IPromptDataProvider.EjecutarPromptAsync()
    └─ Output: PromptResultado
```

### Timeout & Retry Policy

#### Durable Functions automáticamente:

- Timeout por Activity: `RetryOptions.FirstRetryInterval`
- Reintentos configurables en Orchestrator

#### Configuración (appsettings.json):

```
{
  "Pipeline": {
    "ExtraccionTimeoutSeconds": 300,
    "ClasificacionTimeoutSeconds": 60,
    "PromptTimeoutSeconds": 120,
    "RetryMaxAttempts": 3,
    "RetryInitialDelaySeconds": 5
  }
}
```

### En Orchestrator:

```
var retryOptions = new RetryOptions(  
    firstRetryInterval: TimeSpan.FromSeconds(5),  
    maxNumberOfAttempts: 3)  
{  
    Handle = ex => ex is TimeoutException or HttpRequestException  
};  
  
var resultado = await context.CallActivityWithRetryAsync(  
    "ExtraerActivity",  
    retryOptions,  
    input);
```

## Fallback Chain

```
// ConfigurableExtraerDataProvider.cs  
  
// Flujo con fallback:  
try  
{  
    resultadoCu = await _azureProvider.ObtenerDatosAsync(input, cancellationToken);  
  
    if (EsResultadoCuSuficiente(resultadoCu, config, umbral))  
    {  
        return resultadoCu; // ✓ Éxito  
    }  
  
    // Confianza insuficiente → Usar fallback  
    fallbackRazon = "insufficient_extraction:conf=0.45<0.75";  
}  
catch (Exception ex)  
{  
    // Excepción → Usar fallback  
    fallbackRazon = $"exception:{ex.GetType().Name}";  
}  
  
// Fallback: Intentar GPT  
resultadoGpt = await _gptFallbackProvider.ObtenerDatosAsync(input, config, cancellationToken);  
resultadoGpt.FallbackUsado = true;  
resultadoGpt.FallbackRazon = fallbackRazon;  
  
return resultadoGpt;
```

---

## 7. Configuration & Tipología Bindings

---

### Configuración en ConfiguracionJson

Estructura de ConfiguracionJson (almacenado en DB):

```
{
  "Extraction": {
    "Enabled": true,
    "Provider": "azure-content-understanding",
    "Model": "doc-intelligence-v3",
    "MinFieldsRatio": 0.75,
    "CompletenessUmbral": 0.80,
    "ConfianzaUmbral": 0.70,
    "FallbackProvider": "gpt"
  },
  "Classification": {
    "Enabled": true,
    "Provider": "gpt",
    "Model": "gpt-4o",
    "FallbackThreshold": 0.60,
    "NivelClasificacion": 2
  },
  "PromptConfig": {
    "Enabled": true,
    "Template": "Analiza este documento:\n{contenido}",
    "Model": "gpt-4o"
  }
}
```

## Resolución de Provider Efectivo

En `ExtraerActivity` (orquestador resuelve previamente):

```
// Input a Activity ya tiene:
input.ProviderEfectivo =
    instrucciones.Extraction.Provider != "auto"
        ? instrucciones.Extraction.Provider
        : config.Extraction.Provider;

// Si ProviderEfectivo está seteado, toma precedencia
// De lo contrario, ConfigurableExtraerDataProvider usa config.Extraction.Provider
```

## Priority & Weighting

No implementado en plugin system actual, pero patrón fácil de agregar:

```
public class ProviderWeight
{
    public string Provider { get; set; }
    public int Priority { get; set; } // 1=highest
    public double MinConfidence { get; set; }
    public bool Enabled { get; set; }
}

// En ConfiguracionJson:
{
  "ExtractionProviders": [
    { "Provider": "azure-content-understanding", "Priority": 1, "MinConfidence": 0.75 },
    { "Provider": "gpt", "Priority": 2, "MinConfidence": 0.50 }
  ]
}
```

## 8. Testing Plugins

### Unit Tests

```

using Moq;
using Xunit;
using DocumentIA.Functions.Services;
using DocumentIA.Core.Models;

public class MiCustomExtraerProviderTests
{
    [Fact]
    public async Task ObtenerDatosAsync_WithValidInput_ReturnsExtraccionResultado()
    {
        // Arrange
        var logger = new Mock<ILogger<MiCustomExtraerProvider>>();
        var configLoader = new Mock<TipologiaConfigLoader>();
        var blobService = new Mock<IBlobStorageService>();

        blobService
            .Setup(x => x.DescargarAsync(It.IsAny<string>(), It.IsAny<CancellationToken>()))
            .ReturnsAsync(new byte[] { 1, 2, 3 });

        var provider = new MiCustomExtraerProvider(logger.Object, configLoader.Object, blobService.Object);

        var input = new ExtraccionInput
        {
            Tipologia = "Factura",
            Entrada = new ContratoEntrada { Documento = new DocumentoInfo { Key = "test.pdf" } }
        };

        // Act
        var resultado = await provider.ObtenerDatosAsync(input);

        // Assert
        Assert.NotNull(resultado);
        Assert.Equal("MiCustom", resultado.Proveedor);
        Assert.True(resultado.DatosExtraidos.Count > 0);
        Assert.False(resultado.FallbackUsado);
    }

    [Fact]
    public async Task ObtenerDatosAsync_WithNullTipologia_ThrowsArgumentException()
    {
        // Arrange
        var logger = new Mock<ILogger<MiCustomExtraerProvider>>();
        var configLoader = new Mock<TipologiaConfigLoader>();
        var blobService = new Mock<IBlobStorageService>();

        var provider = new MiCustomExtraerProvider(logger.Object, configLoader.Object, blobService.Object);

        var input = new ExtraccionInput { Tipologia = null };

        // Act & Assert
        await Assert.ThrowsAsync<ArgumentException>(() => provider.ObtenerDatosAsync(input));
    }
}

```

## Integration Tests con Orchestrator

```
[Fact]
public async Task ExtraerActivity_CallsProvider_ReturnsResult()
{
    // Arrange
    var logger = new Mock<ILogger<ExtraerActivity>>();
    var provider = new Mock<IExtraerDataProvider>();

    provider
        .Setup(x => x.ObtenerDatosAsync(It.IsAny<ExtraccionInput>(), It.IsAny<CancellationToken>()))
        .ReturnsAsync(new ExtraccionResultado
        {
            Proveedor = "Mock",
            DatosExtraidos = new Dictionary<string, object> { { "test", "value" } }
        });

    var activity = new ExtraerActivity(logger.Object, provider.Object);

    var input = new ExtraccionInput { Tipologia = "Factura" };

    // Act
    var resultado = await activity.Run(input);

    // Assert
    Assert.NotNull(resultado);
    Assert.Equal("Mock", resultado.Proveedor);
    provider.Verify(x => x.ObtenerDatosAsync(It.IsAny<ExtraccionInput>(), It.IsAny<CancellationToken>()), Times.Once);
}
```

## Usando Mock Servers

Para simulaciones de servicios externos:

```
// Usar GptDirectExtraerDataProvider con mock OpenAI
// 1. Configurar en appsettings.json:

{
  "AzureOpenAI": {
    "Endpoint": "http://localhost:8080", // Mock server
    "ApiKey": "mock-key"
  }
}

// 2. Mock server devuelve respuestas JSON predecibles
// 3. Provider mantiene código sin cambios
```

## 9. Ejemplo Completo: Custom Classifier

### Requisitos

Crear un clasificador basado en expresiones regulares + fallback a GPT.

### Implementación

File: `src/backend/DocumentIA.Functions/Services/Classification/RegexClasificarProvider.cs`

```

using DocumentIA.Functions.Abstractions;
using DocumentIA.Core.Models;
using DocumentIA.Core.Configuration;
using Microsoft.Extensions.Logging;
using System.Text.RegularExpressions;

namespace DocumentIA.Functions.Services.Classification;

public class RegexClasificarProvider : IClasificarDataProvider
{
    private readonly ILogger<RegexClasificarProvider> _logger;
    private readonly GptClasificarDataProvider _gptFallback;
    private readonly Dictionary<string, (string Pattern, double Weight)> _patterns;

    public RegexClasificarProvider(
        ILogger<RegexClasificarProvider> logger,
        GptClasificarDataProvider gptFallback)
    {
        _logger = logger;
        _gptFallback = gptFallback;

        // Patrones regex por tipología
        _patterns = new Dictionary<string, (string, double)>
        {
            { "Factura", ("^(Factura|Invoice|INV)", 1.0) },
            { "Presupuesto", ("^(Presupuesto|Quote|Quotation)", 1.0) },
            { "Orden_Compra", ("^(Orden de Compra|PO|Purchase Order)", 1.0) },
            { "Remito", ("^(Remito|Shipping|Packing Slip)", 0.9) }
        };
    }

    public async Task<ResultadoClasificacion> ClasificarAsync(
        ClasificacionInput input,
        CancellationToken cancellationToken = default)
    {
        _logger.LogInformation("Iniciando clasificación Regex");

        var contenido = ExtractTexto(input);
        var (tipologia, confianza) = MatchearPatrones(contenido);

        if (confianza >= 0.7)
        {
            _logger.LogInformation("Clasificación exitosa por regex: {Tipologia} (conf={Confianza})",
                tipologia, confianza);

            return new ResultadoClasificacion
            {
                TipologiaDetectada = tipologia,
                Confianza = confianza,
                Modelo = "regex-v1",
                FallbackLLM = false
            };
        }

        // Fallback a GPT si confianza baja
        _logger.LogWarning("Regex insuficiente (conf={Confianza}). Usando fallback GPT.", confianza);

        var resultadoGpt = await _gptFallback.ClasificarAsync(input, cancellationToken);
        resultadoGpt.FallbackLLM = true;

        return resultadoGpt;
    }

    private (string Tipologia, double Confianza) MatchearPatrones(string contenido)
    {
        var mejorMatch = ("Desconocido", 0.0);

        foreach (var (tipologia, (pattern, weight)) in _patterns)
        {

```



```

        if (Regex.IsMatch(contenido, pattern, RegexOptions.IgnoreCase))
        {
            var confianza = weight;
            if (confianza > mejorMatch.Item2)
            {
                mejorMatch = (tipologia, confianza);
            }
        }
    }

    return mejorMatch;
}

private string ExtractTexto(ClasificacionInput input)
{
    if (!string.IsNullOrEmpty(input.DocumentoBase64Override))
    {
        // Extraer primeras líneas del documento (simplificado)
        return "Factura"; // Tu lógica de OCR aquí
    }

    return input.DatosNormalizados.Values
        .OfType<string>()
        .FirstOrDefault("") ?? "";
}
}

```

## Registrar en Program.cs

```
services.AddSingleton<RegexClasificarProvider>();
```

## Test

```

[Fact]
public async Task ClasificarAsync_WithInvoiceKeywords_ReturnsFactura()
{
    var logger = new Mock<ILogger<RegexClasificarProvider>>();
    var gptFallback = new Mock<GptClasificarDataProvider>();

    var provider = new RegexClasificarProvider(logger.Object, gptFallback.Object);

    var input = new ClasificacionInput
    {
        DatosNormalizados = new Dictionary<string, object>
        {
            { "texto", "Factura N° INV-2024-001" }
        }
    };

    var resultado = await provider.ClasificarAsync(input);

    Assert.Equal("Factura", resultado.TipologiaDetectada);
    Assert.True(resultado.Confianza >= 0.9);
    Assert.False(resultado.FallbackLLM);
}

```

# 10. Troubleshooting

## Plugin no Carga

**Síntomas:** `InvalidOperationException: No implementation of IExtraerDataProvider found`

## Causas & Soluciones:

### 1. No registrado en DI

```
// FALTA en Program.cs:  
services.AddSingleton<MiCustomExtraerProvider>(); // ✓ Agregar
```

### 2. Registrado pero no usado en ConfigurableProvider

```
// Falta en ConfigurableExtraerDataProvider:  
"mi-custom" => await _miCustomProvider.ObtenerDatosAsync(...) // ✓ Agregar
```

### 3. Interfaz incorrecta

```
// INCORRECTO:  
public class MiProvider : ICustomInterface { }  
  
// CORRECTO:  
public class MiProvider : IExtraerDataProvider { }
```

## Plugin Timeout

**Síntomas:** Activity function falla después de 5 minutos

### Causas:

- Operación externa muy lenta (API, BD, red)
- Infinite loop en código del plugin
- Bloqueo de recursos (file locks, concurrencia)

### Soluciones:

```
// 1. Aumentar timeout en appsettings.json
{
  "Pipeline": {
    "ExtraccionTimeoutSeconds": 600 // ↑ De 300 a 600
  }
}

// 2. Agregar logs para debug
_logger.LogInformation("Iniciando llamada externa...");
using var cts = CancellationTokenSource.CreateLinkedTokenSource(cancellationTokens);
cts.CancelAfter(TimeSpan.FromSeconds(120));

try
{
  resultado = await MiOperacionAsync(cts.Token);
}
catch (OperationCanceledException)
{
  _logger.LogError("Operación cancelada por timeout");
  throw;
}

// 3. Implementar retry exponencial
for (int attempt = 0; attempt < 3; attempt++)
{
  try
  {
    return await MiOperacionAsync(cancellationTokens);
  }
  catch (TimeoutException) when (attempt < 2)
  {
    await Task.Delay(1000 * (int)Math.Pow(2, attempt));
  }
}
}
```

## Unexpected Output Format

**Síntomas:** `JsonException: The JSON value could not be converted to...`

### Causas:

- Plugin devuelve schema diferente al esperado
- null donde se espera value type
- Tipo incorrecto (string vs int)

### Soluciones:

```
// X INCORRECTO:
return new ExtraccionResultado
{
  Paginas = null, // Int no puede ser null
  DatosExtraidos = new { Factura = 123 } // Debe ser Dictionary
};

// ✓ CORRECTO:
return new ExtraccionResultado
{
  Paginas = input.Entrada.Documento.Pages ?? 0,
  DatosExtraidos = new Dictionary<string, object>
  {
    { "Factura", 123 }
  }
};
```

## Plugin Fallback Infinito

**Síntomas:** Stack overflow, logs repetitivos

**Causas:**

- Fallback provider también falla
- Configuración circular de providers
- Exception en validación de fallback

**Soluciones:**

```
// 1. Validar en fallback
if (string.IsNullOrEmpty(fallbackRazon))
{
    _logger.LogWarning("Fallback sin razón registrada. Deteniendo.");
    throw new InvalidOperationException("Fallback loop detectado");
}

// 2. Máximo 1 fallback
if (resultado.FallbackUsado)
{
    _logger.LogError("Ya se usó fallback. No reintentar.");
    throw new InvalidOperationException("Fallback ya utilizado");
}

// 3. Provider distinto para fallback
// ✓ Usar: Azure CU → GPT fallback (dos providers diferentes)
// X NO usar: Azure CU → Azure CU fallback (mismo provider)
```

---

## Resumen: Checklist para Crear Plugin

- ☐ Crear clase que implemente `IExtraerDataProvider` / `IClasificarDataProvider` / `IPromptDataProvider`
- ☐ Implementar método único requerido (`ObtenerDatosAsync` / `ClasificarAsync` / `EjecutarPromptAsync`)
- ☐ Retornar tipo de resultado correcto (`ExtraccionResultado` / `ResultadoClasificacion` / `PromptResultado`)
- ☐ Agregar logging en inicio y fin de ejecución
- ☐ Manejar excepciones y propagar con log
- ☐ Registrar en DI: `services.AddSingleton<MiProvider>()`
- ☐ Agregar enrutamiento en Configurable provider: `"mi-custom" => await _miProvider.MethodAsync(...)`
- ☐ Escribir unit tests (input válido, input inválido, exception)
- ☐ Escribir integration tests con Activity
- ☐ Testear con mock servers si aplica
- ☐ Documentar configuración en `ConfiguracionJson`
- ☐ Comitear cambios

---

## Referencias

- [DocumentIA Architecture](#)
- [Technical Design](#)
- [API Documentation](#)
- [Azure Functions Documentation](#)
- [Durable Functions Patterns](#)